Kalpion

Security Features

What protects the data, and what does not.

Who this is for	Anyone deciding whether the tool is safe to put in front of their people, and anyone who has to verify that decision. No security background is assumed.
What it covers	Every control built into the software, grouped by what it protects: keeping organisations apart, signing in, sessions, permissions, what people can see, files, transport, abuse, secrets, auditing and the data lifecycle.
How to read it	Each item says what it does, why it is there, and where in the code it lives. Where a control has limits, the limits are stated: a document that only lists strengths is not useful to anyone who has to rely on it. Section 15 is the honest list of what this does NOT do.
Version	1.0, reflecting the code as of 28 August 2026.

Contents

2. Signing in

Passwords are stored as bcrypt hashes. The original password is never written down anywhere - not in the database, not in a log, not in a backup.

(`src/services/authService.js`)

Minimum length, and a block list. Passwords below the configured minimum are refused, as are the most commonly guessed ones and single repeated characters. A new password must differ from the current one.

Five wrong attempts locks the account for fifteen minutes. The counter is kept in the shared registry, not in the server's memory, so an attacker cannot reset it by hitting a different server, and it survives a restart.

(`src/services/authService.js`, table `login_attempts`)

A correct password does not clear a live lockout. Once locked, the account stays locked for the full window. This is deliberate: an attacker who eventually guesses correctly during a lockout gets nothing.

Sign-in takes the same time whether or not the account exists. When the email is unknown, the server still performs a full bcrypt comparison against a throwaway hash. Without that, the response time alone tells an attacker which email addresses are real - which is how attackers build target lists.

One generic message for every failure. "Invalid credentials" whether the account is unknown, the password is wrong, or the account is deactivated.

Password reset uses a split token. The emailed link carries a public part (used to find the row) and a secret part (compared against a stored hash). The database never holds anything that could be replayed as a reset link. Links expire after one hour, are single-use, and any outstanding links are destroyed when a reset completes. *(`src/services/authService.js`)*

"Forgot password" always reports success. Whether or not the address is registered, the response is the same, so the form cannot be used to test which email addresses exist.

Bulk-imported employees must change their password. People added from a spreadsheet receive a derived temporary password and are blocked from the rest of the application until they replace it - enforced on the server, not merely in the screen. *(`src/middleware/auth.js`, `src/services/userImportService.js`)*

One-time codes, when enabled. Sign-in by phone or email code is available. Codes are hashed, expire in five minutes, allow a limited number of wrong guesses each, and are single-use. The development "write the code to the log" provider refuses to run in production, because a live system that logs sign-in codes has leaked them. *(`src/services/otpService.js`, `smsService.js`)*

3. Sessions

The session token is signed, and carries no secrets. It identifies who is claiming to be signed in and which organisation they belong to.

Every request re-reads the user from the database. The token is treated as a claim, never as a source of truth. This is what makes three things work immediately rather than up to eight hours later:

- deactivating someone ends their session on their next request;

- demoting someone takes their elevated permissions away at once;
- resetting a password invalidates every session opened with the old one.

(`loadLiveUser` in `src/middleware/auth.js`)

Password changes kill old tokens exactly. The token carries the value of the account's `password_changed_at` as it stood when the token was issued. If the stored value has moved on, the token is dead. This is compared value-to-value rather than by timestamp, so it is immune to clock differences between the application server and the database - an earlier timestamp-based attempt let old tokens survive a password change.

The server keeps no session state. It can be restarted, or run on several machines, without signing anybody out.

4. Who can do what

Permissions are enforced on the server. Every route declares the roles that may reach it, and the service behind it re-checks. A screen that hides a button is a convenience, never the control.

(`src/routes/*.js`, and the `assertOrgAdmin` style guards in the services)

The service layer never sees the web request. Business rules take a database connection and a user object, so a rule cannot accidentally depend on who is calling it, and every rule is directly testable.

Nobody reviews their own idea. Enforced in the service, not the screen.

Approval routes are data, not scattered conditions. Reviewer roles, final approver roles, the stage list and the committee threshold are configuration. An unrecognised value is dropped rather than stored, so a typo cannot create a stage nobody holds - which would strand every idea that reached it.

(`src/services/settingsService.js`, `approvalStages.js`)

The organisation's administrator is always a final approver. So an idea can never get stuck with nobody able to close it.

5. Not showing people what they should not see

This is where most of the recent work has gone, because in an ideation tool the data being protected is the idea itself.

Full idea text is never sent to a browse list. Not to employees, not to managers, not to administrators. The All Ideas table, the Idea Board and the leaderboard all carry a short summary; the full text comes only from the detail request, and only to someone entitled to it. This closes a real class of leak: text that is in the response but not on the screen is invisible to anybody testing by looking. *(`redactSolution` in `src/services/ideaService.js`)*

Who may read a full proposal is a per-organisation setting. Authors and reviewers; managers only; or everyone. The author always reads their own idea, in every mode - a setting that hid someone's own writing from them would be a bug rather than a policy.

The problem statement is held back too. A well-written problem statement often contains the whole insight, so an uninvolved colleague sees an extract whose length the organisation sets, not the whole write-up.

Section-by-section control. An organisation chooses exactly what a colleague outside an idea may read: the one-line proposal, the problem extract, the benefits, the business case, the attachments, the discussion, the co-suggesters, the approval history. The default is the one-line proposal alone. The title, code, status, department, impact and score are never hidden, because they are what makes an idea findable and what the leaderboard counts.

(`src/services/ideaSections.js`)

The rule is enforced in every place ideas appear - the detail overlay, the Idea Board, the browse list and the comments endpoint - rather than in one screen.

Anonymous means anonymous. Masking the header is not enough: the approval timeline names an actor on every entry, starting with the submitter's own "Submitted" row, and the co-suggester list names the people who raised it with them. All three are masked together. *(`src/services/ideaService.js`)*

The machine's written assessment is governed separately. Everyone can always vote; who may read the automatic reasoning is its own setting, defaulting to managers. Authors always see the assessment of their own idea.

Exports carry only what the person may see, and say so. The single-idea PDF is built from the same filtered view as the screen, so opening the export to everybody did not open the data. There are two documents: whoever raised the idea, the colleagues credited on it and the people reviewing it get the full two-page closure record; everybody else gets a one-page summary sheet carrying the title, who raised it, where it has got to, and a one-line gist.

The second document exists for a reason beyond redaction. Handing a bystander the closure form with two pages of emptied boxes looks like a broken export and invites the reader to wonder what was removed. A sheet that says "summary" on its face is both safer and more honest, and it is stamped with the name of the person who exported it, so a leaked copy is traceable.

(`src/controllers/exportController.js`, `ideaPdfService.js`)

The screens do not offer what the server will not give. A colleague outside an idea is shown a "Summary" button rather than "View" - opening the full overlay would produce a title and a row of locked notices. The button is chosen from `viewer_inside`, computed on the server by one function (`isInsideIdea`), which also drives how much text is sent and which sections are stripped. Answering the same question in three places is how the three answers drift apart. *(`src/services/ideaService.js`)*

Drafts are private. An unsubmitted draft and its attachments are visible only to their author and the organisation's administrators.

6. On-screen deterrents

Screen guard on every page that shows ideas. It hides idea text the moment the browser window loses focus - which is the first thing most capture tools do - lays the reader's own name, employee number and the time across the page, blanks the content for printing and print-to-PDF, and clears the clipboard after PrintScreen. On by default; an organisation can switch it off.

(`frontend/src/components/ScreenGuard.jsx`)

Stated plainly: this is a deterrent, not a control. No web page can stop a phone camera pointed at a monitor, a screen recorder started before the page loaded, or a second machine mirroring the display. What it does is make casual capture awkward and make any leak attributable. The actual protection is section 5: the text is never sent to people who should not have it.

7. Handling what users send us

Every query is parameterised. User-supplied values are always bound, never concatenated into SQL. The places where a fragment is built as text build only *structure* - a column name chosen from a fixed pair, a list of `?` placeholders, a set of whitelisted column names, a date filter picked from a fixed map.

There is one deliberate exception, stated because a blanket claim would be false: row limits. MySQL 8 refuses `LIMIT` and `OFFSET` as bound parameters, so three paginated queries build the number into the statement. Each one passes through `parseInt` and a clamp on the line above, so only a plain integer can ever reach the string - a text value cannot survive the journey. This is checked by the assurance suite, which fails the build if any service binds a row limit (the portability trap that made the admin user list return 500 on the production database while passing every local test).

Settings are an explicit allowlist. A setting not named in the list cannot be written, so a new form field is inert until it is deliberately accepted. Values are then validated: modes against their vocabulary, numbers clamped to a range, role and stage lists filtered against the catalogue.

(`src/services/settingsService.js`)

Request bodies are capped at 1 MB, and the text fields on an idea have their own length limits.

Registration details are format-checked, on the server as well as in the browser - Udyam, GSTIN, PAN, CIN, NIC code, PIN code, phone, email domain. The browser checks are courtesy; the server checks are the rule.

Corporate email only for self-registration. Applications from free consumer mailboxes are refused, because a company cannot be verified from a Gmail address. *(`src/services/registrationService.js`)*

Output is escaped. React escapes by default, and the two places that build HTML as text - the print-to-PDF windows - escape explicitly, because React's protection does not apply to string concatenation.

CSV exports are quoted with doubled internal quotes, so a company named `O"Brien, Inc.` cannot break a column boundary.

8. File attachments

- Extension allowlist on upload and again on download.
- Stored under a server-generated random filename, never the name the user

supplied, so a crafted filename cannot become a path.

- Per-file size ceiling, set by the organisation and clamped by a platform

maximum the organisation cannot exceed.

- Per-organisation total storage quota, measured from the directory on disk

rather than a running total, because a counter drifts the moment a file is removed by hand.

- No file is reachable by URL. Every download goes through a checked handler

that confirms the requester's organisation, the idea's state, and the resolved path.

- The content type served comes from a fixed map, not from the upload.

(`src/services/uploadService.js`)

9. Transport and browser headers

- **HTTPS enforced in production**, with HSTS (one year, including subdomains).
- **Security headers** via Helmet: content security policy locked to nothing

(the API serves JSON and downloads, never HTML), `frame-ancestors: none` so it cannot be framed, `Referrer-Policy: no-referrer`, no `X-Powered-By`.

- **Cross-origin requests are allowlisted.** Only configured origins are

accepted, and only the headers the application actually uses.

- **Database connections use TLS** where the provider supplies a certificate,

and verify it. *(`src/app.js`, `src/config/index.js`)*

10. Rate limiting and abuse

- **Per-address global cap**, tunable so a large office behind one connection is

not throttled.

- **A much tighter cap on sign-in, reset and forgot-password.** The per-account

lockout stops someone grinding one account; it does nothing against one guess each against a thousand accounts. This is what caps that. Only failures count against the budget.

- **Expensive operations** (rescoring, large exports) have their own hourly cap.
- **Per-organisation request allowances, from the plan.** A bigger plan buys

more of the platform. The allowance is sized from the plan's user cap at roughly 15,000 requests per permitted user per month - about thirty times what ordinary use costs - and a number set on a particular organisation overrides it. Counted in the shared registry rather than in memory, so it is not reset by a deploy and is correct across several servers.

Three safeguards sit around it, because an earlier flat cap of 2,000 a month was applied to ordinary page loads and took a live customer offline: a grace band above the line where the customer is warned rather than refused; a warning at 80% so they hear it from us first; and an allowlist - sign-in, support, notifications, branding and settings always answer, so an organisation at its limit can still get in, see why, and raise a ticket. Reaching a limit is a commercial conversation, not an outage. *(`src/middleware/tenantQuota.js`, `src/services/planService.js`)*

11. Secrets and configuration

The server refuses to start in production if it is configured insecurely. It checks, and crashes with an explanation rather than running wide open, when:

- the signing secret is missing, still the example placeholder, or too short -

anyone with the repository could otherwise forge an administrator token for any organisation;

- the database password is empty, or the database user is `root`;

- the allowed origins still include localhost;
- the frontend URL is missing, points at localhost, or is plain `http` - which

would put password-reset links on an unencrypted address.

(`validateConfig` in `src/config/index.js`)

Stored secrets are never returned. The mail password and the QCMS integration key are write-only from the interface: the server returns a mask and a "one is set" flag, never the value, and writes a new one only when a real value is supplied - so saving an untouched form cannot wipe a working key.

(`src/services/settingsService.js`, `integrationService.js`)

Nothing sensitive is logged. Passwords, tokens and keys do not appear in logs or in error responses. Internal errors return a generic message; the detail stays server-side.

12. Audit and accountability

Every decision on an idea is recorded. Submission, assignment, each reviewer's decision, escalation, approval, rejection, archiving, patentability - each with who did it, when, and any note. Shown on the idea itself and on the Audit Trail screen. *(`idea_workflow`, `src/services/reportService.js`)*

Sign-ins are recorded separately from lockout state. The lockout counter is cleared on every successful sign-in and so can never answer "who signed in, and when". The activity record is append-only, kept for 180 days, and then trimmed. It captures the time, the outcome (signed in, wrong password, locked out), the address, the browser and device, and an approximate location.

(`src/services/activityService.js`)

Location is derived from the browser's own time zone, not from the IP address. Looking the address up would mean sending our administrators' addresses to a third-party service on every sign-in, and behind a hosting provider's proxy the address is a private one that no lookup could resolve anyway. A time zone is a band of the world and follows whatever the machine is set to, so it is treated as a hint, not evidence.

The platform console's activity page lists IFQM staff only. Customers' internal staff movements are their business.

Metering and logging fail open. A fault in counting or recording is written down and ignored rather than becoming an outage - but an unsafe *configuration* fails closed and stops the server. Running insecurely is worse than not running; a counting fault is not.

13. Data lifecycle

- **Archiving is not deletion.** Archived ideas and tickets leave the everyday

lists but keep their points, history and recorded savings, and can be restored. Bulk archiving is reversible by the same operation.

- **Comments are soft-deleted when they have replies**, so a thread does not

lose its shape, and are removed outright when they do not.

- **Deleting an organisation requires typing its code**, and dropping its

database is a separate, explicit opt-in.

- **Schema changes are a deliberate human step.** Deploying new code never

alters the database. Migrations are forward-only, recorded in a ledger, and idempotent. Automatic schema changes on deploy are how data gets lost. *(`db/migrations/`, `backend/scripts/migrate.js`)*

- **Backups** are scripted separately. *(`backend/scripts/backup.js`)*

14. What is checked automatically

The test suite is part of the security story, because a control nobody verifies is a control that quietly stops working.

- 33 integration tests covering the tenant boundary, session invalidation,

lockout behaviour, redaction and the upload rules.

- A 228-case assurance run covering authentication, authorisation, data

protection, reliability, scalability and recovery - including deliberately adversarial cases: five wrong passwords then the correct one, failed sign-ins counted on one server locking the account on another, and cross-organisation reads by id.

15. Known limits

Stated plainly, because pretending otherwise would make the rest of this document less trustworthy.

- **Screenshots cannot be prevented.** Section 6 explains what the guard does

and does not do.

- **Location is approximate.** It reflects a machine's time-zone setting, not a

place.

- **A determined authorised reader can still copy what they are shown.** Anyone

entitled to read an idea can retype it. The controls reduce reach and make copying attributable; they do not make a trusted reader untrusted.

- **Attachment contents are not scanned.** Files are checked by extension and

size and are never executed or served as HTML, but there is no virus scanning. An organisation handling untrusted uploads should scan at the storage layer.

- **Encryption at rest is the database provider's.** The application relies on

the hosting provider's disk encryption rather than encrypting columns itself.

- **Two-factor authentication is available as a one-time code**, but is not

mandatory and is off until an SMS provider is configured.

Prepared for IFQM. Reflects the code as of 10 August 2026.